

Conversational Booking Agent

Architecture Reference — *agent-service*

Deterministic State Machine · Local LLM for NLU · Platform-Grounded Facts

This document describes the system as it currently stands, the same relationship *docs/architecture.md* has to *docs/design.md* for the rest of the platform. For the chronological build log and open gaps see *docs/agent_service_progress.md*.

Contents

1. Design Philosophy
2. Module Map
3. The State Machine
4. One Turn — End to End
5. Structured Options (Clickable Buttons)
6. Chat-to-Browser Booking Hand-off
7. What the LLM Is Never Trusted With
8. Current Gaps and Where to Look Next

1. Design Philosophy

The agent is **not** a general reasoning LLM loosely calling tools. It is a **deterministic state machine** that owns every fact and every decision. A local LLM (Llama 3.2 3B via Ollama) is called at most **twice per turn**, for two narrow, bounded jobs:

NLU — `nlu.py`

- 1 Free text → a flat JSON dict of candidate field values. Never resolves anything against the platform, never writes an id, never decides what to say next.

Articulation — `responder.py`

- 2 The state machine's already-decided template text → the same facts, reworded for tone. Instructed not to add/remove/invent a fact; falls back to the literal template text on any failure.

Every fact in a response — which city matched, what date is real, what a showtime costs — is produced by plain Python validating against the real platform. The LLM can misfire (return nothing, echo a few-shot example, mangle a proper noun) without ever causing the agent to hallucinate a booking detail. At worst it asks the same question again or sounds slightly stiff, because the literal template text is always a safe fallback.

2. Module Map

The table below lists every file in `services/agent-service/` and its single responsibility. Files highlighted in blue touch the LLM.

File	Responsibility
<code>main.py</code>	FastAPI app. GET /health, POST /message — the only route. All three turn kinds (free text / button click / out-of-band booking status) are dispatched here.
<code>nlu.py</code>	LLM call #1. <code>extract(message)</code> → dict — {city, movie, theatre, date, count, showtime}, raw text only, temperature=0. Fails closed (all-None) on any error.
<code>resolution.py</code>	Deterministic, stdlib-only (difflib). <code>resolve(entities)</code> → dict — re-categorises/normalises a misrouted NLU fragment against the platform's real combined city+movie+theatre name pool.
<code>dialogue_manager.py</code>	The state machine itself: <code>DialogueState (ABC)</code> , one subclass per state, <code>DialogueStateMachine (dispatcher)</code> , <code>Orchestrator</code> (turn sequencing + cross-state correction-clearing).
<code>responder.py</code>	LLM call #2. <code>articulate(template_text)</code> → str — rephrase-only, temperature=0.3. Falls back to the literal input on any failure.
<code>context.py</code>	<code>BookingContext</code> dataclass — <code>session_id</code> , <code>user_id</code> , <code>city_id</code> , <code>movie_id</code> , <code>date</code> , <code>theatre_id</code> , <code>showtime_id</code> , <code>count</code> , <code>seat_ids</code> , <code>booking_id</code> . Resolved values only — never written directly from free text.
<code>states.py</code>	<code>State(str, Enum)</code> — six identifiers, no behaviour.
<code>session_store.py</code>	In-memory dict[str, tuple[State, BookingContext]] + a lock, alive for the process lifetime. No expiry.
<code>platform_client.py</code>	Thin httpx wrapper: <code>list_cities</code> , <code>list_movies</code> , <code>list_theatres</code> , <code>list_showtimes_for_movie</code> , <code>get_booking</code> . Raises <code>PlatformUnavailableError</code> on timeout/non-2xx.
<code>config.py</code>	Pure leaf module (import os only) — every env-derived value and magic number.

■ These files are the only two LLM touch-points in the codebase.

3. The State Machine

Six states, each owning exactly one BookingContext slot. The machine walks the **full priority list from GREETING every turn**, regardless of which state the session is parked at. This is what makes correcting an earlier slot from a later point in the conversation work uniformly — no special correction path exists.



State responsibilities and validation sources:

State	Owns	Validates against
GREETING	city_id	list_cities()
COLLECTING_MOVIE	movie_id	list_movies(city_id) — currently-playing only
COLLECTING_DATE	date	Real calendar dates in list_showtimes_for_movie(movie_id, city_id)
COLLECTING_THEATRE	theatre_id	Theatres in list_showtimes_for_movie(...) actually screening the movie
COLLECTING_SHOWTIME	showtime_id	Showtimes at theatre_id, narrowed by date
AWAITING_BOOKING	booking_id	get_booking(booking_id) — written by main.py only, not from entities

Orchestrator.process() — how a turn walks the states:

The orchestrator always walks the full priority list from GREETING, every turn. Each state's resolved-check works identically for a first answer and a correction — matched-and-unchanged is a silent pass-through; matched-and-different is a correction. The walk stops at the first state that isn't resolved this turn.

Cascading clears:

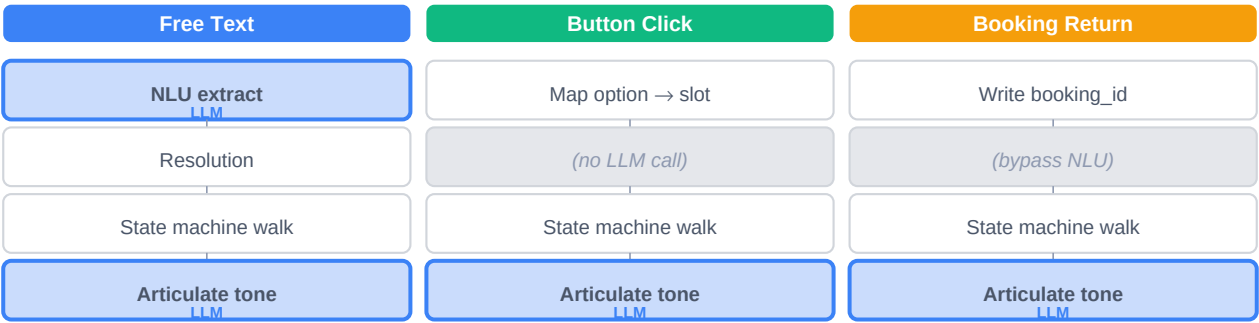
Correcting an upstream slot invalidates whatever was picked downstream. This is handled in the orchestrator (not each state) to preserve one-slot-one-responsibility.

city_id	movie_id, date, theatre_id, showtime_id, booking_id
movie_id	date, theatre_id, showtime_id, booking_id
date	theatre_id, showtime_id, booking_id
theatre_id	showtime_id, booking_id
showtime_id	booking_id

Note: date is cleared on a city_id/movie_id correction because list_showtimes_for_movie's real dates are scoped by city_id+movie_id — a date picked against the old movie/city may not exist for the new one.

4. One Turn — End to End

POST /message is the only route. Every turn is exactly one of three kinds:



Shaded / LLM-labelled steps call Ollama (Llama 3.2 3B). Max 2 calls per turn (free-text path), 1 on button/booking paths. Neither call decides a fact. Up to two LLM calls per turn (free-text path), one (articulate only) on the button/booking_id paths. Neither call ever decides a fact or calls a platform API.

Handle return shape:

Each state's handle(context, entities) returns (resolved, message, options, link_data).

Field	Meaning
resolved	True once this state's slot is settled this turn; the orchestrator may move forward.
message	This state's own text only, empty when nothing changed.
options	Exact real values (city names, movie titles, formatted labels) for UI to render as clickable buttons. A single candidate is still presented as a one-item list — no state auto-resolves on count of one.
link_data	Only populated by AwaitingBookingState; a hand-off URL merged into response.extra after articulation runs — never passed through responder.articulate().

5. Structured Options (Clickable Buttons)

Every unresolved state's response carries options: list[str] — the exact real values it is enumerating in its prose. A UI renders these as buttons; clicking one sends selected_option instead of typed text.

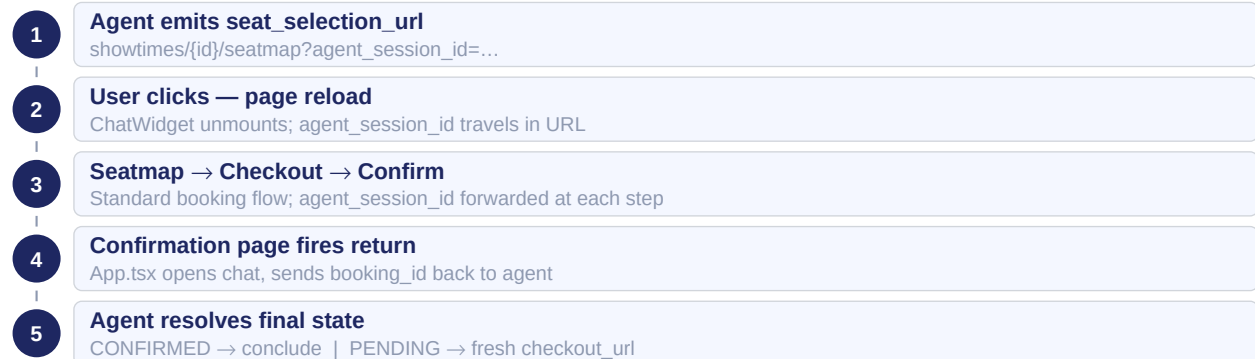
main.py detects selected_option and calls dialogue_manager.entities_from_selected_option(state_before, selected_option), which writes the literal clicked text straight into the slot the previous turn's options belonged to — no LLM call, no ambiguity.

State	Writes to field
GREETING	city
COLLECTING_MOVIE	movie
COLLECTING_DATE	date
COLLECTING_THEATRE	theatre
COLLECTING_SHOWTIME	showtime

This is the only path that ever resolves COLLECTING_SHOWTIME in practice — there is no NLU field or parsing for an arbitrary typed time-of-day phrase. Buttons are the deterministic alternative once date-narrowing leaves more than one candidate. Every other state's slot can be corrected by either a click or typed free text, handled identically by the state's own match-and-write logic.

6. Chat-to-Browser Booking Hand-off

Seat selection and payment are **not** reimplemented in chat. `AWAITING_BOOKING` hands off to customer-web's existing seatmap → checkout → confirmation pages with a real link, then learns the outcome via a return-to-chat redirect that only fires when the flow was actually started from the chat. A customer browsing those pages directly sees zero behaviour change.



AwaitingBookingState response logic by booking status:

booking_id / status	Response
None (never handed off)	Offer seat_selection_url
PlatformUnavailableError	"temporarily unavailable," no link
None returned (404 / unknown id)	Clear booking_id, offer seat_selection_url again
CONFIRMED	Conclude — no link, nothing left to do
PENDING	Offer checkout_url
EXPIRED / CANCELLED	Clear booking_id, re-offer seat_selection_url with explanatory prefix

App.tsx lives outside `<Routes>` and never unmounts across client-side navigation, but a hard reload (the checkout → confirmation step) destroys it. Continuity rides in the URL query string (agent_session_id / agent_booking_id), not in memory.

7. What the LLM Is Never Trusted With

This section states the core safety guarantee explicitly.

Facts

F

Every name, id, date, and price in a response comes from a real platform call or `BookingContext`, never from either LLM call's own text.

Decisions

D

Which state runs next, whether a match is valid, what to ask — all plain Python against real platform data.

Structural data

S

A hand-off URL is computed by `AwaitingBookingState` and travels through `link_data`, never through `responder.articulate()`. Articulation only ever sees prose meant to be reworded, never a string where exact characters matter.

**Session identity**

`context.session_id` and `booking_id` are written by `main.py` from the request itself, never extracted from a message.

This means a flaky or adversarial model response can **degrade the conversation** (ask again, sound stiff, miss an extraction) but **cannot** fabricate a booking detail, leak a different session's data, or smuggle a fact past the deterministic layer.

8. Current Gaps and Where to Look Next

This document describes the architecture *as built*. It is not a complete account of known limitations, in-flight NLU reliability issues, or what the next session should pick up.

Gap	Reference	Detail
NLU reliability	<code>docs/agent_service_progress.md</code>	In-flight reliability issues and known failure modes are documented in the chronological build log.
Session TTL	<code>session_store.py</code>	The in-memory session store has no expiry. Sessions live for the process lifetime.
Showtime free-text	<code>dialogue_manager.py</code>	Arbitrary typed time-of-day phrases ("6pm", "the early show") are not parsed. <code>COLLECTING_SHOWTIME</code> is button-only by design.
LLM backend options	<code>config.py</code> / <code>nlu.py</code>	Ollama (Llama 3.2 3B) is the current backend. Claude API and ChatGPT are candidate alternatives under evaluation.
Session store scaling	<code>session_store.py</code>	The in-memory dict is single-process only. A Redis-backed store is needed for multi-process/multi-instance deployment.